

Inheritance, Interfaces, Packages

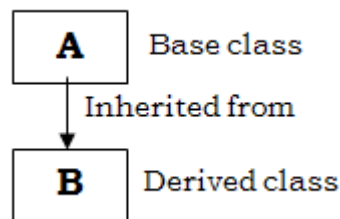
Introduction:

- We know that, inheritance is one of the most important object-oriented concept and Java language supports it.
- The main concept behind inheritance is reusability of code (Software) is possible by adding some extra feature into existing software without modifying it.
- In java language, '**Object**' is super class of all classes even your own defined class also.

Definition:

- The mechanism of creating new class from existing class is called as "Inheritance".
- The existing or old class is called as 'Base class' or 'Parent class' or 'Super class' and new class is called as 'Derived class' or 'Child class' or 'Sub class'.
- While deriving the new class from exiting one then, the derived class will have some or all the features of existing class and also it can add its own features i.e sub class will acquire features of base class without modifying it.
- When a new class is derived from exiting class, then all public, protected and default data of base class can easily accessed into its derived class where as private data of base class cannot inherited i.e. this private data cannot accessed in derived class.

Consider following example:



In above figure class B is inherited from class A. Therefore, class A is called as *Super class* or *parent class* or *base class* of class B whereas class B is called as *Sub class* or *child class* or *derived class*.

Need or Features of Inheritance:

- 1) Inheritance allows reusability of code.
- 2) It is always nice to use existing software instead of creating new one by adding some extra features without modifying it. This can be done by only inheritance.
- 3) Due to inheritance software cost is reduces.
- 4) Due to inheritance software developing time is also reduces.
- 5) Due to inheritance, we can add some enhancement to the base class without modifying it this is due to creating new class from exiting one.

Syntax to derive new class from existing class: (syntax for inheritance)

```
class    derived_class_name    extends    base_class_name
{
    // Declaration of Variables and definition of methods.
}
```

Here;

`class` is keyword to define class.

`extends` is keyword used to create or derive or extends new class.

`derived_class_name` is name of newly created class which is an identifier.

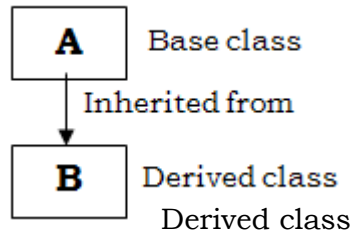
`base_class_name` is name of existing class which is also an identifier.

Types (forms) of Inheritance:

Depending upon number of base classes and number of derived classes and their arrangement, inheritance has following types.

1) Single Inheritance:

- In case of Single inheritance there is only *one base class from which we derive only one new class*.
e.g.



From above figure we can write single inheritance as:

```
class    A
{
    //Declaration of Variables and definition of methods.
}

class    B  extends    A
{
    //Declaration of Variables and definition of methods.
}
```

Following program shows implementation of single inheritance:

```
import    java.util.Scanner;
class    A
{
    int    x,y;
    Scanner    sc=new    Scanner(System.in);
    void    get( )
    {
        System.out.println("Enter two numbers ");
        x=sc.nextInt( );
        y=sc.nextInt( );
    }
}
```

```
class    add    extends    A
{
    int    z;
    void    doadd( )
    {
        super.get( );
        z=super.x+super.y;
        System.out.println("Addition="+z);
    }
}

public    class    single
{
    public    static    void    main(String [ ] args)
    {
        add    p=new    add( );
        p.doadd( );
    }
}
```

‘super’ keyword:

- In Java, **super** to refer to **the parent (superclass) class** from a child class.
- We use ‘super’ keyword along with members of super class **when we want to access parent class members into its subclass when they are hidden or overridden.**

Use of ‘super’ keyword:

- ‘super’ keyword is used to access members of base class into its derived class.
- We can access, instance variables of base class into its derived class using syntax:

```
super.variable;
```

- We can access, methods of base class into its derived class using syntax:

```
super.method(arg1,arg2, ...);
```

- Note that, we need not call the default constructor of super class because it is defaultly available to its sub class. And hence, we can made call to parameterized constructor of base class into its derived class using syntax:

super(arg1,arg2,.....);

Consider following example that demonstrate use of 'super' keyword:

```
class base
{
    int x;
    base()
    {
        x= 10;
    }
}
class derived extends base
{
    int x;
    derived()
    {
        x= 20;
    }
    void display()
    {
        System.out.println("Value of x from base: "+super.x);
        System.out.println("Value of x is from derived: "+x);
    }
}
class superDemo
{
    public static void main(String[] args)
    {
        derived d= new derived();
        d.display();
    }
}
```

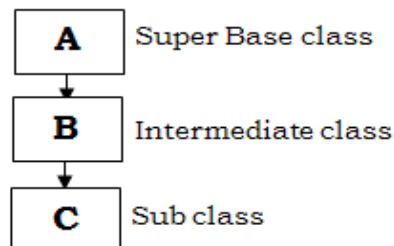
OUTPUT:

Value of x from base: 10
Value of x is from derived: 20

2) Multilevel Inheritance:

- In case of multilevel inheritance, we can derive new class from another derived classes, further from derived classes we again derive new class and so on.
- Also, there are some levels of inheritances therefore it is called as “Multilevel inheritance”

E.g.



From above figure we can write multilevel inheritance as follow:

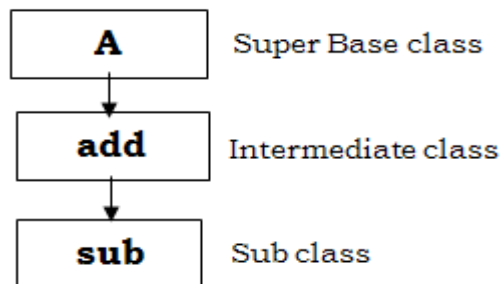
```

class    A
{
    //Declaration of Variables and definition of methods.
}
class    B  extends  A
{
    //Declaration of Variables and definition of methods.
}
class    C   extends  B
{
    //Declaration of Variables and definition of methods.
}

```

Implementation of Multilevel Inheritance is as follow:

Figure:



```

import java.util.Scanner;
class A
{
    int a,b;
    void get()
    {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter two no=");
        a=sc.nextInt();
        b=sc.nextInt();
    }
}
class add extends A
{
    int c;
    void doadd()
    {
        super.get();
        c=super.a+super.b;
        System.out.println("Addition="+c);
    }
}

```

```

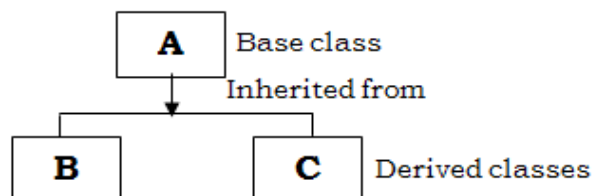
class sub extends add
{
    int c;
    void dosub()
    {
        super.doadd();
        super.get();
        c=super.a-super.b;
        System.out.println("Subtraction="+c);
    }
}
public class Multilevel
{
    public static void main(String []args)
    {
        sub t=new sub();
        t.dosub();
    }
}

```

3) Hierarchical Inheritance:

- In case of hierarchical inheritance there is *only one base class from that class we can derive multiple new classes.*
- The base class provides all its features to all derived classes which are common to all sub classes.
- The hierarchical inheritance comes into picture when certain feature of one level is shared by many other derived classes.

e.g.



From above figure we can write hierarchical inheritance as fallow:

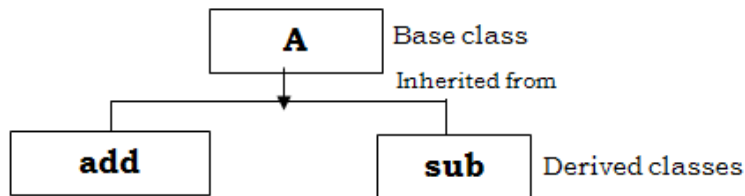
```

class A
{
    // Declaration of Variables and definition of methods.
}
class B extends A
{
    // Declaration of Variables and definition of methods.
}
class C extends A
{
    // Declaration of Variables and definition of methods.
}

```

Implementation of Hierarchical Inheritance is as follow:

Figure:

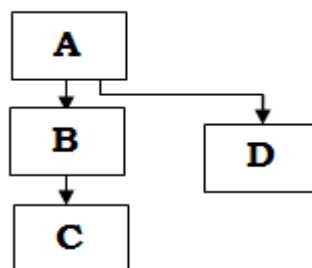


<pre> import java.util.Scanner; class A { int a,b; void get() { Scanner sc=new Scanner(System.in); System.out.println("Enter two No="); a=sc.nextInt(); b=sc.nextInt(); } } class add extends A { int c; void doadd() { super.get(); c=super.a+super.b; System.out.println("Addition="+c); } } </pre>	<pre> class sub extends A { int c; void dosub() { super.get(); c=super.a-super.b; System.out.println("Subtraction="+c); } } public class Hierarchical { public static void main(String []args) { add p=new add(); p.doadd(); sub q=new sub(); q.dosub(); } } </pre>
---	--

4) Hybrid Inheritance:

- Hybrid inheritance is the *combination of two or more forms of inheritances*.
- To implement hybrid inheritance, we have to combine two or more forms of inheritances and such a combination of different forms of inheritances is called "Hybrid inheritance".

E.g.



The above inheritance is a combination of multilevel and hierarchical inheritances therefore it is "Hybrid Inheritance".

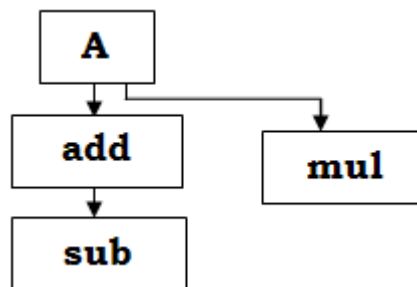
From above figure we can write hybrid inheritance as follow:

```

class A
{
    // Declaration of Variables and definition of methods.
}
class B extends A
{
    // Declaration of Variables and definition of methods.
}
class C extends B
{
    // Declaration of Variables and definition of methods.
}
class D extends A
{
    // Declaration of Variables and definition of methods.
}

```

Implementation of Hybrid Inheritance is as fallow:



```

import java.util.Scanner;
class A
{
    int a,b;
    void get()
    {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter two No=");
        a=sc.nextInt();
        b=sc.nextInt();
    }
}
class add extends A
{
    int c;
    void doadd()
    {
        super.get();
        c=super.a+super.b;
        System.out.println("Addition="+c);
    }
}
class sub extends add
{
    int c;
    void dosub()
    {
        super.doadd();
        super.get();
        c=super.a-super.b;
        System.out.println("Subtraction="+c);
    }
}

```

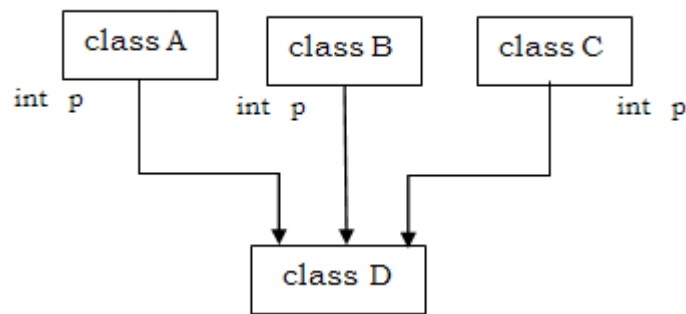
```

class multi extends A
{
    int c;
    void domulti()
    {
        super.get();
        c=super.a*super.b;
        System.out.println("Multiplication="+c);
    }
}
public class Hybrid
{
    public static void main(String []args)
    {
        sub p=new sub();
        p.dosub();
        multi q=new multi();
        q.domulti();
    }
}

```

5) Multiple Inheritance:

- Multiple inheritance means deriving or creating *only one sub class from multiple super classes*.
- But **Java does not directly support for multiple inheritance**. This multiple inheritance leads confusion for the programmer, which is shown in following figure:



- In above fig, Super classes i.e. *Class A*, *Class B* and *Class C* have same instance variable 'int p' and from these classes *Class D* is derived or extended. In this case, *Class D* has confusion regarding with copy of 'int p'. The *Class D* does not know whose copy of 'int p' is accessed.
- Also, **class D extends A, B, C** such **syntax is invalid** in Java.
- If Java does not support for 'Multiple inheritance' then it will not consider as pure OOP language.
- In fact, Java is pure OOP language. Because, multiple inheritance can be implemented using 'interface' concept. So, let's see the 'Interface' concept in details.

Interface

Introduction:

We know that, Java language does not support for multiple inheritance but **we can implement multiple inheritance using 'interface'**.

Interface:

- An 'interface' is **similar to the class which is used to achieve abstraction and multiple inheritance** in java.
- All the methods of interface by default **public & abstract**. (*abstract method* does not have a body i.e. no implementation)
- Also, 'interface' has variables but by default all interface variables are **public, static and final**
- An interface contains only abstract methods which are all incomplete therefore it is not possible to create object of interface.
- But, we can create separate classes from interface where we can implement all the methods of interface. These classes are called as 'implementation' classes.
- Since, 'implementation classes' contains method definition therefore we can create object of implementation classes.
- Every implementation class can have its own implementation of abstract methods of the interface.

Syntax to define interface:

```
interface Interface_name
{
    datatype    var1=value;
    ⋮
    ⋮
    return_type    Method1();
    ⋮
    ⋮
}
```

here,

'interface' is keyword used to define interface.

'Interface_name' is name of interface which is an identifier.

Syntax to define new class (implementation class) from interface:

```
class    implement_class_name    implements    Interface_name
{
    datatype    var1, varN;

    public return_type    Method1( )
    {
        -----
        -----
    }

    public return_type    MethodN( )
    {
        -----
        -----
    }
}
```

here,

‘implements’ is keyword used to define or create new class from interface.

‘Interface_name’ is name of interface from which *implement_class_name* is created.

Characteristic or properties of Interface:

- 1) An ‘interface’ is a *specification of method prototype* only i.e. these methods does not have any body or definition.
- 2) An interface will have zero or more abstract methods which are all by default public & abstract.
- 3) An interface can have variables which are by default **public, static and final**. This means all variables of interface are constant therefore we have to assign them a value.
- 4) None of the methods in interface can be private, protected or static.
- 5) We **cannot** create an object of interface but we can create reference to interface.

E.g.

```
interface    one
{
    void    get();
}
class    demo
{
    public    static    void    main(String ar[])
    {
        one    obj=new    one();    //invalid due to creation of object of interface
        one    t;    //valid due to creation of reference of interface
    }
}
```

- 6) All methods of interface should be defined in its implementation classes. If any method is not implemented then that implementation class should be declared as ‘abstract’
- 7) An interface **can extend** another interface.

E.g.:

```
interface    one
{
    void    get();
}
interface    two    extends    one
{
}
}
```


- 8) An interface **cannot implement** another interface.

E.g.:

```
interface one
{
    void get();
}
interface two implements one //it is invalid
{
}
```

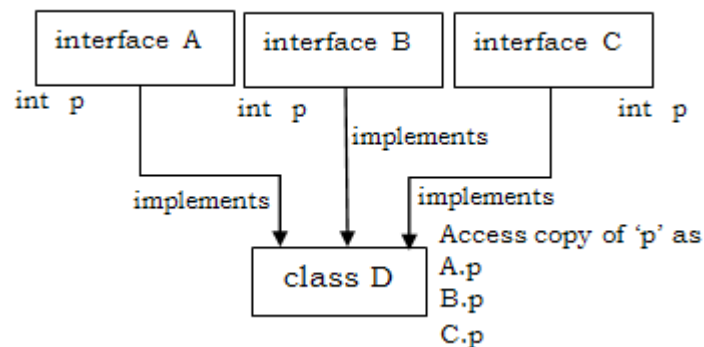
- 9) A class can implement (not extends) from multiple interfaces which helps to implement multiple inheritance.

E.g.:

```
class MyClass implements interface1, interface2, interface3
{
}
}
```

Multiple Inheritance using Interfaces:

- Multiple inheritance means deriving or creating only one sub class from multiple super classes.
- But, Java does not support for multiple inheritance. And that multiple inheritance leads confusion for programmer, which is shown in following figure:



- In above fig, *class D* is implementation class which is implemented from three interfaces viz. *interface A*, *interface B* and *interface C*. In this case, *Class D* can access individual copy of '*int p*' from all interfaces using *A.p*, *B.p* and *C.p*. Because, by default all variables of interface are static therefore they are accessed by interface name. Thus, confusion regarding copy of '*int p*' in *Class D* is solved.
- Also, *class D implements A, B, C* such syntax is valid in Java.
- Thus, In Java, multiple inheritance is implemented using 'interface' concept.

Following program shows multiple inheritance using interface.

<pre>interface A { int p=10; void calculate(); } interface B { int p=20; void calculate(); } interface C { int p=30; void calculate(); }</pre>	<pre>class D implements A,B,C { public void calculate() { int z; z=A.p+B.p+C.p; System.out.println("Addition="+z); } } class multiple { public static void main(String s[]) { D obj=new D(); obj.calculate(); } }</pre>
--	--

In above program:

class D is implementation class which is implemented from interfaces A,B and C.

The *calculate()* method in implementation 'class D' must be defined with '**public**' modifier because it overrides its abstract methods of interfaces A,B,C. And by default all interface methods are public.

If do not define it with public modifier then compiler gives compilation error.

Method Overriding:

- In Java, Runtime polymorphism is achieved by 'method overriding'.
- When super class & sub class have same methods with same signatures i.e. both super class and sub class have same method prototype then sub class method overrides (replaces) the super class method such a concept is called as 'Method overriding'
- We know that, super class methods are also accessed by object of sub class. But, in method overriding both classes has same methods with same signatures in such situation; JVM only executes methods of sub class i.e. here sub class method replaces (overrides) the super class method and hence super class method not executes.
- Thus, we can say that, super class method is overridden by sub class method. And hence super class method not executed by sub class object.
- Consider following example which shows Method overriding concept:

```
class first
{
    void get(int p)
    {
        -----
        -----
        -----
    }
}
class second extends first
{
    void get(int x)
    {
        -----
        -----
        -----
    }
}
```

- In above example, 'first' is super class whereas 'second' is sub class of 'first' class.
- Also, both classes has get() method with same signatures. And if we made call to *get()* method using object of sub class then only sub class *get()* method is invoked i.e. sub class method overrides the base class method & get() method of 'first' class never executed. Hence, here method overriding is done.

Following program demonstrate the Method overriding concept using instance method:

<pre>class one { void add() { System.out.println("I am in Base class"); } } class two extends one { void add() { System.out.println("I am in Derived Class"); } }</pre>	<pre>class all { public static void main(String []ar) { two t=new two(); t.add(); } }</pre> OUTPUT: I am in Derived Class
---	--

Homework: Write difference between method overloading and method overriding.

Dynamic Method Dispatch or Runtime Polymorphism in Java:

- We know that, **Runtime polymorphism** is achieved in Java by 'method overriding'.
- Dynamic method dispatch is the mechanism by which **a call to an overridden method is resolved at run time**, rather than compile time.
- In dynamic method dispatching, **superclass reference variable can refer to a subclass object**. This is also known as "upcasting". Java **uses this fact (Upcasting) to resolve calls to overridden methods at run time**.
- When an overridden method is called through a superclass reference, Java determines which version (superclass/subclasses) of that method is to be executed based upon the type of the object being referenced at the time call occurs. Thus, this determination is made at run time.
- Therefore, if a superclass contains a method that is overridden by a subclass, then different types of objects are referenced through a superclass reference variable.
- Following program shows Dynamic method dispatch concept:

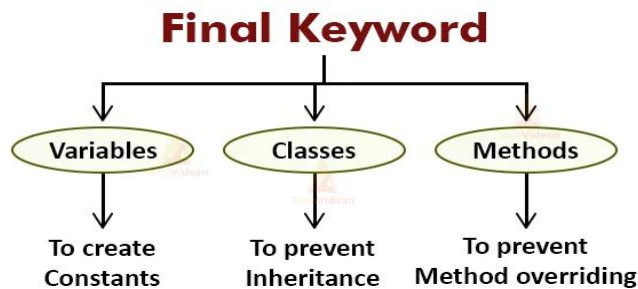
<pre>class A { void show() { System.out.println("A class method"); } } class B extends A { void show() { System.out.println("B class method"); } } class C extends A { void show() { System.out.println("C class method"); } }</pre>	<pre>class Dish { public static void main(String ar[]) { A ref; //Super class reference A a=new A(); ref=a; //'ref' reference to Class A ref.show(); // call to Class A method B b=new B(); ref=b; //'ref' reference to Class B ref.show(); // call to Class B method C c=new C(); ref=c; //'ref' reference to Class C ref.show(); // call to Class C method } } OUTPUT: A class method B class method C class method</pre>
--	--

Difference between method overloading and method overriding:

Method Overloading	Method Overriding
1) Defining multiple method <u>with same name with different signature</u> is called 'Method overloading'	1) Defining multiple method <u>with same name with same signature</u> is called 'Method overriding'
2) Method overloading in Java is a form of <u>compile-time polymorphism</u> .	2) Method overriding in Java is a form of <u>run-time polymorphism</u> .
3) Method overloading is done within same class.	3) Method overriding is done within different classes i.e. in super class & sub class.
4) In method overloading, return type of methods can be same or different.	4) In method overriding, return type of methods <u>must be same</u> .
5) JVM decides which method has to be called depending upon <u>parameters of methods</u> .	5) JVM decides which method has to be called depending upon <u>object of sub class or super class</u> .
6) Method overloading is <u>code development</u> . Same methods are developed to perform same task depending on parameters.	6) Method overriding is <u>code replacement</u> . The sub class method overrides (replaces) the super class method.

‘final’ Keyword:

- The ‘final’ keyword is a modifier used with classes, class attributes and class methods, which makes them non-changeable i.e. make them as constant.
- We can use ‘final’ keyword *before variable declaration or before method definition or before class definition*. We explain all these as follows:



1) final Variable:

- If we use ‘final’ keyword before variable declaration then that variable becomes ‘final’ variable and that final variables are now treated as constant in Java.
- That is, to declare constant in Java, ‘final’ keyword is used.
- *final variable* cannot be modified because they treated as constant.

Syntax:

```
final    datatype    name=value;
```

here,

‘final’ is keyword.

‘datatype’ is valid data type in java.

‘name’ is an identifier which is constant name .

‘value’ is any constant value assigned to final variable.

Ex: final double PI= 3.14;

Here, Incrementation or Decrementation or initialization other value to ‘PI’ is invalid because it is ‘final’ and not modified.

2) final Method:

- If we use ‘final’ keyword before method definition then that method becomes ‘final’ method.
- And final methods cannot override by its derived or sub class. That is sub class method *cannot replace* code of its base class *final method*.
- *Note that: ‘final’ methods of base class can be accessible into its derived class.*

Syntax:

```
final    return_type    Method_name( )
{
    -----
    -----
    -----
}
```

Note:

- *Final methods of base class cannot override into its derived class therefore method overriding using final methods are not possible. That is final methods are not overridden.*
- *But, ‘final’ methods of base class can be accessible into its derived class.*

Program:

Program:- 1)

```
class A
{
    int a;
    final void get()
    {
        a=10;
    }
}
class B extends A
{
    int b;
    void get()
    {
```

Program:- 2)

```
class A
{
    int a;
    final void get()
    {
        a=10;
    }
}
class B extends A
{
    void show()
    {
```

<pre> b=50; } } public class FinalMethod { public static void main(String []args) { B obj=new B(); obj.get(); } } </pre>	<pre> super.get(); //access final method in sub class System.out.println("Value="+super.a); } } public class FinalMethod { public static void main(String []args) { B obj=new B(); obj.show(); } } </pre>
Above program gives compile error as: get() in B cannot override get() in A Since, final method cannot override.	Above program successfully executes. Since, here is no method overriding concept.

3) final Class:

- If we use 'final' keyword before class definition then that class becomes 'final' class.
- And final class prevents inheritance. That is, *we are not able to create or extends new class from 'final class'*
- If we don't want to allow sub class to access features of its base class then base class has to made as 'final'

Syntax:

```

final    class    class_name
{
    -----
    -----
    -----
}

```

Ex.

```

final    class    A
{
    -----
    -----
}
class B extends A    // Is invalid.....
{
}

```

Program:- 1) <pre> final class A { void get() { System.out.println("Base class"); } } class B extends A { void show() { super.get(); System.out.println("Derived class"); } } public class FinalClassDemo { public static void main(String []args) { </pre>	Program:- 2) <pre> class A { void get() { System.out.println("Base class"); } } class B extends A { void show() { super.get(); System.out.println("Derived class"); } } public class FinalClass { public static void main(String []args) { </pre>
--	---

<pre> B obj=new B(); obj.show(); } } </pre>	<pre> B obj=new B(); obj.show(); } } </pre>
Above program gives compile error as: cannot inherit from final A Since, final class cannot inherited.	Above program successfully executes. Since, here no final class.

Abstract Method and Abstract class

Abstract Method:

- Abstract method is such method **that does not have any definition related to class** (it has no body.)
- Abstract method should be declared with keyword '**abstract**'
- Abstract method contains **only method header i.e. method prototype.**
- Abstract method has no body therefore they are also called 'incomplete method'
- When abstract method is **declared into super class then it is compulsory to re-define it into its sub classes.**
- When **method is declared as 'abstract' then respective class should also be declared as 'abstract'**
- An abstract method is written when same method has to perform different tasks depending upon object requirements.

Ex:

```

abstract class ABC
{
    abstract void get( );
}

```

- In above example, get() method is declared as abstract using keyword 'abstract'. And that method does not have any definition.

Abstract class:

- Abstract class is such class that declared with keyword 'abstract' and that **contains zero or more abstract methods.**
- Abstract class may also contain **instance variable & concrete methods.**
- We **cannot create object of Abstract class** i.e. **Abstract class cannot instantiated.** Because abstract class contains incomplete abstract methods.
- But **we can create reference of Abstract class.** And that *reference* may refers to *object of its sub classes.*
- Abstract class may have constructors. Then, question is that who invoke constructor of abstract class? Since, object of abstract class cannot be created.
- Answer is that: constructors of abstract class is invoked by object of its sub classes.
- We cannot declare a class as final and abstract at a time. Because, *final* keyword prevents to create sub classes and *abstract* keyword allows super class to have sub classes.

Ex:

```

abstract class ABC
{
    abstract void get( );
    void show( )    //concrete method.
    {
        -----
        -----
    }
}

```

- In above example, class ABC is declared as abstract using keyword 'abstract'. And that contains get() abstract method and show() concrete method.

Following program demonstrate the use of abstract class and abstract methods.

<pre> abstract class MyBase { abstract void calculate(double x); } class Square extends MyBase { void calculate(double a) { System.out.println("Square="+a*a); } } </pre>	<pre> class Cube extends MyBase { void calculate(double a) { System.out.println("Cube="+a*a*a); } } class abstractDemo { </pre>
---	---

<pre> } } class squareRoot extends MyBase { void calculate(double a) { System.out.println("SquareRoot="+Math.sqrt(a)); } } </pre>	<pre> public static void main(String as[]) { Square t1=new Square (); squareRoot t2=new squareRoot (); Cube t3=new Cube (); t1.calculate(5); t2.calculate(36); t3.calculate(4); } </pre>
---	--

Difference between Abstract class and Interface:

Abstract Class	Interface
1) An abstract class contains abstract methods and also some concrete methods.	1) An interface only contains abstract methods.
2) An abstract class contains instance variable	2) An interface cannot contain instance variables. It contains only constants.
3) By default, abstract methods of abstract class are not public.	3) By default, methods of interface are public and abstract.
4) All abstract methods of abstract class should be implemented into its sub classes.	4) All abstract methods of interface should be implemented into its implementation classes.
5) Abstract class is declared using ‘abstract’ keyword.	5) Interface is declared using ‘interface’ keyword.
6) Methods in abstract class can be private, protected or static.	6) Methods in interface cannot be private, protected or static.
7) Abstract class may have ‘constructors’	7) Interface does not have ‘constructors’

Packages:

- A java package is a **directory or folder which is collection or group of classes, interfaces and sub-packages.**

Advantages/Features/Properties of Java Package:

- Avoids Name Conflicts** – Packages **allow classes with the same name to exist in different packages without conflict.**
- Provide Better Organization** – Packages **group related classes and interfaces together, making large projects easier to manage.**
- Improves Code Readability** – Well-structured packages **make the code easier to understand and maintain.**
- Provide Access Protection (Security)** – Packages **provide access control** (default, protected, public, private) to restrict unwanted access.
- Make Code Reusability** – Classes in a **package can be reused in different programs by importing the package.**
- Provide Modular Programming** – Packages help to **divide applications into smaller, manageable modules.**
- Easy Maintenance** – **Errors can be identified and fixed easily because related classes are grouped together.**
- Supports Hierarchical Structure** – **Packages can contain sub-packages**, allowing better project structure.

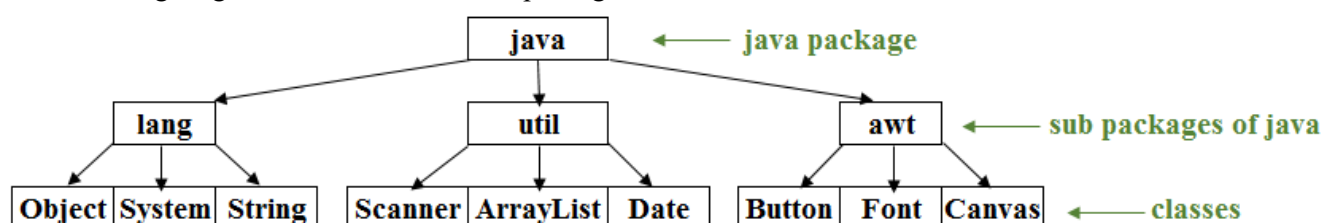
Types of Packages-

- Packages are divided into two categories:
 - 1) Built-in Packages (packages from the Java Library)
 - 2) User-defined Packages / Custom package (create your own packages)

Let's see these types in details-

1) Built-in Packages:

- Built-in packages are those **packages which are predefined in Java Library (in JDK).**
- The packages in Java library **contains methods which are useful for managing input, database programming, and much more.**
- While doing programming in Java, we can either import a single class (along with its methods and attributes), or a whole package that contains all the classes. Such as java.util.Scanner or java.util.*
- There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.
- Following diagram shows some built-in packages:



- To use a class or a package from the library, we use the **import** keyword as follow:

```
import package.name.Class;    // Import a single class
import package.name.*;       // Import the whole package
```

Import a single Class:

- We can import single class as follow:

```
import java.util.Scanner;    // Import a single class
```

In the example above, java.util is a package, while Scanner is a class of the java.util package.

Import a whole package:

We can import whole java.util package as follow:

```
import java.util.*;    // Import whole util package
```

2) User defined Packages: (Custom package)

- The packages which are **created or defined by user is called "User defined packages"**.
- To **create** user defined package, **package** keyword is used as follow:

```
package packageName;
```

- We create user defined package **mypack** as follow:

```
package mypack;
public class MypkgClass
{
    public void showMsg()
    {
        System.out.println("This is my package!");
    }
}
```

Compiling user defined package:

Syntax:

```
D:\2101>javac -d directoryWithPath JavaSourceProgram.java
```

The **-d** specifies 'directory' which is the destination for **to save package and its class file**.

We can use any directory name, like d:\2525 etc. at **directoryWithPath**

or, if we want to keep the package within the **same directory then use the dot sign "." at directory**.

Example: Above **MypkgClass.java** source file can be compiled as-

```
D:\2101>javac -d . MypkgClass.java
```

- This forces the compiler to create the "mypack" package within same directory.
- After successful compilation of given program, a new folder was created, called "mypack" that contains MyPkgClass.

Note: The package **name should be written in lower case** to avoid conflict with class names.

Use or importing above created package in program:

```
import mypack. MypkgClass;
class Usepack
{
    public static void main(String []ar)
    {
        MypkgClass obj=new MypkgClass();
        obj.showMsg();
    }
}
```

OUTPUT: *(Compile and Run this program we will get output)*
This is my package!

Example-2:

Creating and importing User defined package:

- **Step 1:** Create java source program with following code:

```
package mypack;
public class all
{
    public double add(double a, double b)
    {
        return(a+b);
    }
    public double sub(double a, double b)
    {
        return(a-b);
    }
    public double multi(double a, double b)
    {
        return(a*b);
    }

    public double div(double a, double b)
    {
        return(a/b);
    }
}
```

- **Step 2:** Save above program with **all.java** in your local directory.
- **Step 3:** Compile program as: d:\2201>javac -d . all.java
After compiling above program, it would create **mypack** package (folder) in your local directory with **all.class** byte code.
- **Step 4:** Now to use this package in our program. Create new java source program and import **mypack** package as-

```
import mypack.all;
class UseDemo
{
    public static void main(String []arg)
    {
        all m=new all();
        double p=m.add(15,4);
        double q=m.sub(55,7);
        double r=m.multi(5,4);
        double s=m.div(65,7);
        System.out.println("Add="+p);
        System.out.println("Sub="+q);
        System.out.println("Multi="+r);
        System.out.println("Division="+s);
    }
}
```

- **Step 5:** Now compile above program as- d:\2201>javac UseDemo.java
- **Step 6:** Run above program as- d:\2201>java UseDemo

Wrapper classes:

- The wrapper class in Java provides the mechanism to convert primitive data types into object and object into primitive data types.
- Wrapper class uses autoboxing and unboxing feature to convert primitives into objects and objects into primitives automatically.
- The automatic conversion of primitive data types into an object is known as "autoboxing" whereas conversion of object into primitive type is called "unboxing".

Use of Wrapper classes in Java:

- Java is an object-oriented programming language, so we need to deal with objects many times like objects in Collections, Serialization, Synchronization, etc. where we need to use the wrapper classes.
- Change the value of variable in Method: **Java by default supports only call by value.** So, if we pass a primitive value to method, it will not change its original value inside method. But, if we convert the primitive value in an object and pass to method, then it will change its original value.
- In **Serialization**, we need to convert the objects into streams to perform the serialization. Here, such conversion done through the wrapper classes.
- In **Synchronization (Multithreading)**, we work with objects for that wrapper classes are used.
- **java.util package**: The java.util package provides the utility classes to deal with objects hence wrapper classes are used.
- **Collection Framework**: Java collection framework works with objects only. All classes of the collection framework (ArrayList, LinkedList, Vector, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, etc.) deal with objects only hence wrapper classes are used.
- Following eight classes of the java.lang package are known as ‘wrapper classes’ in Java. The list of eight wrapper classes is given below:

Primitive Type	Wrapper class
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

Autoboxing:

- The automatic conversion of primitive data type into its corresponding wrapper class is known as “Autoboxing”. For example, byte to Byte, char to Character, int to Integer, long to Long, float to Float, boolean to Boolean, double to Double, and short to Short referred as Autoboxing.
- Wrapper class Example: Primitive to Wrapper

```
//Java program to convert primitive data types into objects
public class AutoBox
{
    public static void main(String args[])
    {
        int      a=20;
        boolean  b=false;
        Integer   x=a;    //autoboxing, compiler will write Integer.valueOf(a) internally
        Boolean   y=b;    //autoboxing
        System.out.println("Integer val="+x);
        System.out.println("Boolean Val="+y);
    }
}
```

Unboxing:

- The automatic conversion of wrapper type into its corresponding primitive type is known as “unboxing”.
- It is the reverse process of Autoboxing. Wrapper class Example: Wrapper to Primitive

```
//Java program that converts object to primitive data types
public class UnBox
{
    public static void main(String args[])
    {
        Integer a=new Integer(3);
        int     x=a;    //unboxing, compiler will write a.intValue() internally
        Boolean b=new Boolean(false);
        boolean y=b;
        System.out.println("int="+x);
        System.out.println("boolean="+y);
    }
}
```

Theory Assignment No: 03

- 1) What is Inheritance? Write need or features of inheritance.
- 2) Explain Single inheritance with example.
- 3) Explain Multilevel inheritance with example.
- 4) Explain Hierarchical inheritance with example.
- 5) Explain Hybrid inheritance with example.
- 6) How multiple inheritance creates confusion or problem in Java?
- 7) What is Interface? List out characteristics or properties of interface.
- 8) How multiple inheritance is implemented using interface in Java?
- 9) Explain Method overriding with example.
- 10) What is Dynamic method dispatch? Explain with example.
- 11) Write difference between method overloading and method overriding.
- 12) Write difference between Abstract class and interface.
- 13) What is wrapper class? Write its use.
- 14) What is Autoboxing and Unboxing? Explain with example.
- 15) Write short note on:
 - a) 'final' keyword
 - b) 'super' keyword
 - c) Abstract method
 - d) Abstract class
 - e) Packages

Practical Assignment No: 06

- 1) Write a program to implement single inheritance.
- 2) Write a program to implement multilevel inheritance.
- 3) Write a program to implement hierarchical inheritance.
- 4) Write a program to implement hybrid inheritance.
- 5) Write a program to implement multiple inheritance using interface.
- 6) Write a program that demonstrate use of method overriding.
- 7) Write a program that demonstrate dynamic method dispatch.
- 8) Write a program that demonstrate use 'final' keyword.
- 9) Write a program that demonstrate Autoboxing concept.
- 10) Write a program that demonstrate Unboxing concept.
- 11) Write a program that creates user defined package and use it in new java source program.
- 12) Write a program demonstrate the use of abstract class and abstract methods
- 13) Write a Java program to create an abstract class Animal with abstract methods eat() and sleep(). Create subclasses Lion, Tiger, and Deer that extend the Animal class and implement the eat() and sleep() methods differently based on their specific behavior.
- 14) Write a Java program to create a class called Shape with methods called getPerimeter() and getArea(). Create a subclass called Circle that overrides the getPerimeter() and getArea() methods to calculate the area and perimeter of a circle.
- 15) Write a Java program to create an interface Flyable with a method called fly_obj(). Create three classes Spacecraft, Airplane, and Helicopter that implement the Flyable interface. Implement the fly_obj() method for each of these three classes.
- 16) Write a Java program to create a base class Sports with a method called play(). Create three subclasses: Football, Basketball, and Rugby. Override the play() method in each subclass to play a specific statement for each sport.

MCQ with Answers

- | | |
|---|--|
| <p>1. Inheritance in Java is mainly used for</p> <ul style="list-style-type: none">- Data hiding- Code reusability- Data binding- Compilation speed <p>2. Which class is the super class of all classes in Java?</p> <ul style="list-style-type: none">- Main- Object- Parent- Root <p>3. Which keyword is used to derive a class from another class?</p> <ul style="list-style-type: none">- inherit- implements- extends- super <p>4. Which members of a base class are NOT inherited?</p> <ul style="list-style-type: none">- Public- Protected- Default- Private <p>5. Which type of inheritance has only one base class and one derived class?</p> <ul style="list-style-type: none">- Multilevel- Multiple- Single- Hybrid <p>6. Which inheritance is not supported directly by Java?</p> <ul style="list-style-type: none">- Single- Multilevel- Hierarchical- Multiple <p>7. Multiple inheritance in Java is implemented using</p> <ul style="list-style-type: none">- Class- Object- Package- Interface <p>8. Which keyword is used to access base class members?</p> <ul style="list-style-type: none">- this- super- base- parent <p>9. Which inheritance allows deriving a class from another derived class?</p> <ul style="list-style-type: none">- Single- Hierarchical- Multilevel- Multiple <p>10. All methods of an interface are by default</p> <ul style="list-style-type: none">- Private- Protected- Public and abstract- Static | <p>11. Which keyword is used to implement an interface?</p> <ul style="list-style-type: none">- extends- implements- interface- inherit <p>12. Method overriding occurs when</p> <ul style="list-style-type: none">- Same name, different signature- Same name, same signature- Different name- Constructor override <p>13. Dynamic method dispatch is also known as</p> <ul style="list-style-type: none">- Compile-time polymorphism- Static binding- Runtime polymorphism- Method overloading <p>14. Which keyword prevents method overriding?</p> <ul style="list-style-type: none">- static- abstract- final- super <p>15. A final class</p> <ul style="list-style-type: none">- Can be inherited- Cannot be inherited- Can be abstract- Can override methods <p>16. An abstract method</p> <ul style="list-style-type: none">- Has body- Is private- Has no body- Cannot be overridden <p>17. Which keyword is used to declare abstract class?</p> <ul style="list-style-type: none">- final- static- abstract- interface <p>18. Packages in Java are used to</p> <ul style="list-style-type: none">- Increase speed- Avoid name conflicts- Compile code- Debug <p>19. Conversion of primitive to object is called</p> <ul style="list-style-type: none">- Unboxing- Casting- Autoboxing- Serialization <p>20. Which wrapper class is used for int?</p> <ul style="list-style-type: none">- Double- Integer- Float- Long |
|---|--|